

EXPERIMENT NO. 4

ADVANCED DATA VISUALIZATION USING MATPLOTLIB AND SEABORN

Name : Dev Sharma

UID : CU24250269

Course : B.Tech CSE

Section : CSE "B"

Roll No. : 17

AIM

To create and analyze different data visualizations using Matplotlib and Seaborn to identify trends, patterns, relationships and insights from the Netflix dataset.

GITHUB LINK :- [https://github.com/DevSharma1912/Mastery-in-Data-Analytics-and-Visualizations-and-Career-Advancement-Technical-/tree/main/Mastery%20in%20Data%20Analytics%20and%20Visualizations%20and%20Career%20Advancement%20\(Technical\)/Experiment4](https://github.com/DevSharma1912/Mastery-in-Data-Analytics-and-Visualizations-and-Career-Advancement-Technical-/tree/main/Mastery%20in%20Data%20Analytics%20and%20Visualizations%20and%20Career%20Advancement%20(Technical)/Experiment4)

STEP 1: IMPORT LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

STEP 2: SELECT DATASET FROM COMPUTER

```
from google.colab import files
import io

uploaded = files.upload()

if not uploaded:
    print("No file selected.")
else:
    for filename in uploaded.keys():
        # Assuming the uploaded file is a CSV
        df = pd.read_csv(io.StringIO(uploaded[filename].decode('utf-8')))
        print("Dataset loaded successfully!")
        print("Shape:", df.shape)
        break # Assuming only one file is uploaded or we take the first one
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving netflix_titles.csv to netflix_titles.csv
Dataset loaded successfully!
Shape: (8807, 12)

STEP 3: DATA PREPROCESSING

```
# Convert date_added into datetime
if "date_added" in df.columns:
    df["date_added"] = pd.to_datetime(
        df["date_added"],
        errors="coerce"
    )

df["year_added"] = df["date_added"].dt.year

# Extract numeric value from duration
if "duration" in df.columns:
    df["duration_num"] = pd.to_numeric(
        df["duration"].str.extract(r"(\d+)")[0],
        errors="coerce"
    )
```

STEP 4: DISPLAY BASIC INFORMATION

```
print("\nColumns:")
print(df.columns.tolist())

print("\nMissing Values:")
print(df.isnull().sum())
```

Columns:
['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating', 'duration', 'listed_in', 'description', 'year_added', 'duration_num']
Missing Values:
show_id 0
type 0
title 0
director 2634
cast 825
country 831
date_added 98

```
release_year    0
rating          4
duration        3
listed_in       0
description     0
year_added      98
duration_num    3
dtype: int64
```

STEP 5: BAR CHART - MOVIES VS TV SHOWS

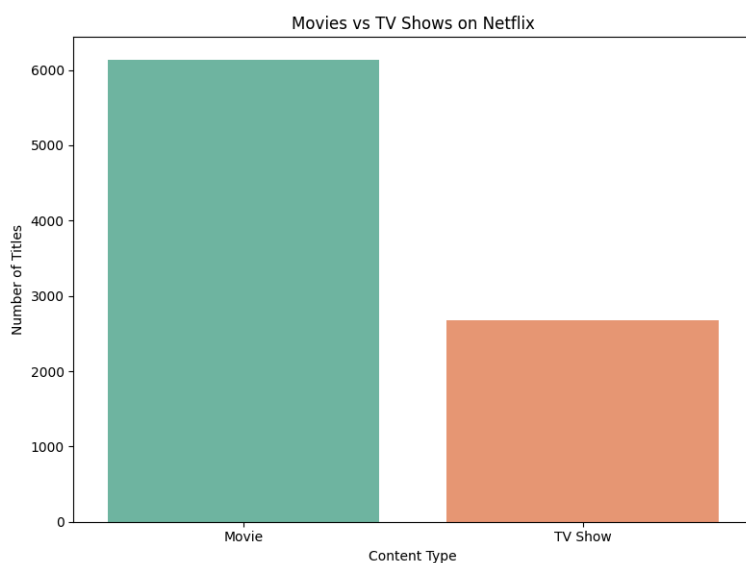
```
if "type" in df.columns:

    type_count = df["type"].value_counts()

    plt.figure(figsize=(8, 6))

    sns.barplot(
        x=type_count.index,
        y=type_count.values,
        hue=type_count.index,
        palette="Set2",
        legend=False
    )

    plt.title("Movies vs TV Shows on Netflix")
    plt.xlabel("Content Type")
    plt.ylabel("Number of Titles")
    plt.tight_layout()
    plt.show()
```



STEP 6: BAR CHART - CONTENT BY RATING

```
if "rating" in df.columns:

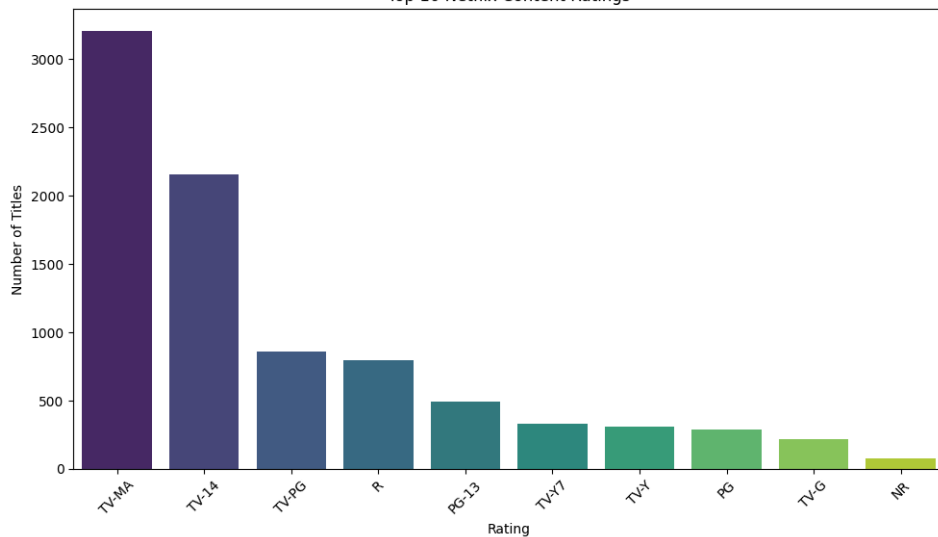
    rating_count = df["rating"].value_counts().head(10)

    plt.figure(figsize=(10, 6))

    sns.barplot(
        x=rating_count.index,
        y=rating_count.values,
        hue=rating_count.index,
        palette="viridis",
        legend=False
    )

    plt.title("Top 10 Netflix Content Ratings")
    plt.xlabel("Rating")
    plt.ylabel("Number of Titles")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

Top 10 Netflix Content Ratings



STEP 7: LINE CHART - CONTENT ADDED BY YEAR

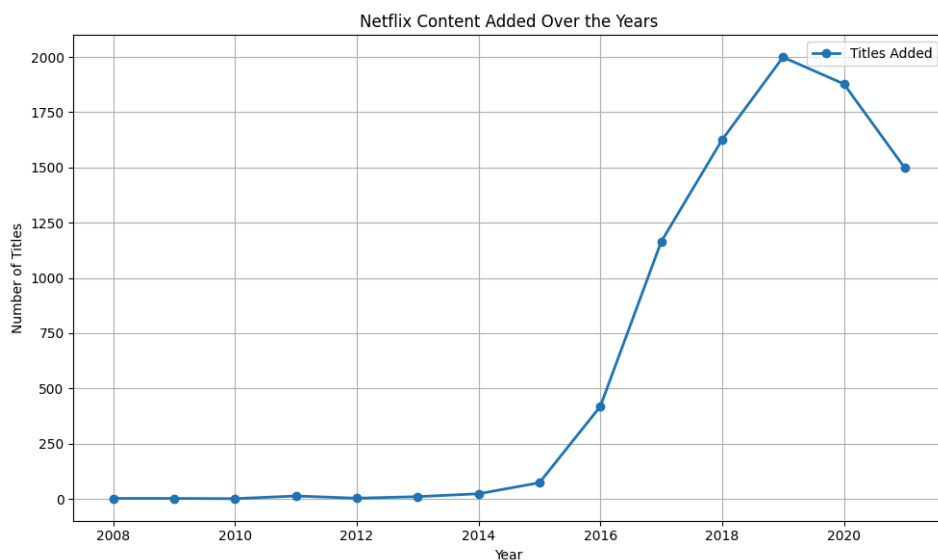
```
if "year_added" in df.columns:

    yearly_content = (
        df["year_added"]
        .dropna()
        .astype(int)
        .value_counts()
        .sort_index()
    )

    plt.figure(figsize=(10, 6))

    plt.plot(
        yearly_content.index,
        yearly_content.values,
        marker="o",
        linewidth=2,
        label="Titles Added"
    )

    plt.title("Netflix Content Added Over the Years")
    plt.xlabel("Year")
    plt.ylabel("Number of Titles")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```



STEP 8: HISTOGRAM - RELEASE YEAR DISTRIBUTION

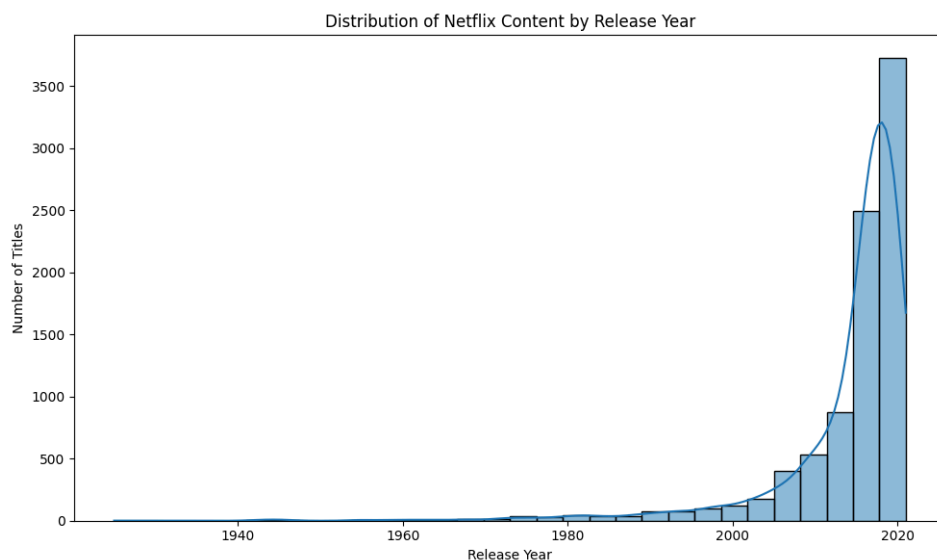
```
if "release_year" in df.columns:

    plt.figure(figsize=(10, 6))

    sns.histplot(
        df["release_year"].dropna(),
        bins=30,
        kde=True
    )

    plt.title("Distribution of Netflix Content by Release Year")
```

```
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()
```



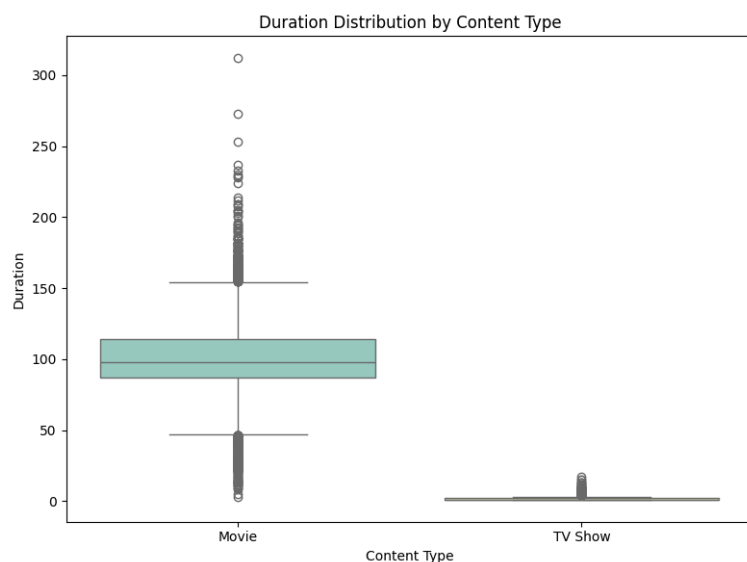
STEP 9: BOX PLOT - DURATION BY CONTENT TYPE

```
if "type" in df.columns and "duration_num" in df.columns:
```

```
plt.figure(figsize=(8, 6))
```

```
sns.boxplot(
    data=df,
    x="type",
    y="duration_num",
    hue="type",
    palette="Set3",
    legend=False
)
```

```
plt.title("Duration Distribution by Content Type")
plt.xlabel("Content Type")
plt.ylabel("Duration")
plt.tight_layout()
plt.show()
```



STEP 10: SCATTER PLOT - RELEASE YEAR VS DURATION

```
if "release_year" in df.columns and "duration_num" in df.columns:
```

```
scatter_data = df[
    ["release_year", "duration_num", "type"]
].dropna()
```

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(
    data=scatter_data,
    x="release_year",
    y="duration_num",
    hue="type",

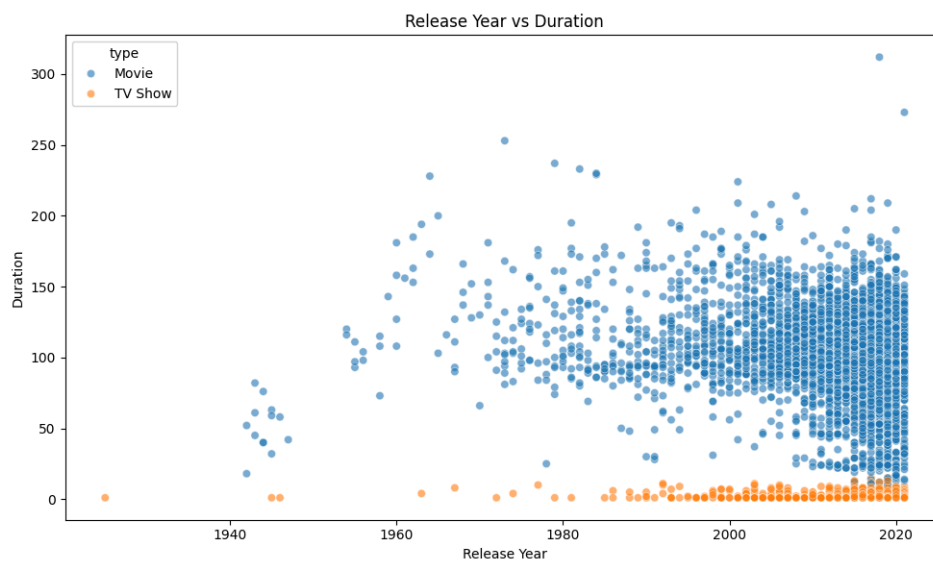
```

```

alpha=0.6
)

plt.title("Release Year vs Duration")
plt.xlabel("Release Year")
plt.ylabel("Duration")
plt.tight_layout()
plt.show()

```



STEP 11: CORRELATION HEATMAP

```

correlation_columns = [
    "release_year",
    "year_added",
    "duration_num"
]

correlation_columns = [
    column for column in correlation_columns
    if column in df.columns
]

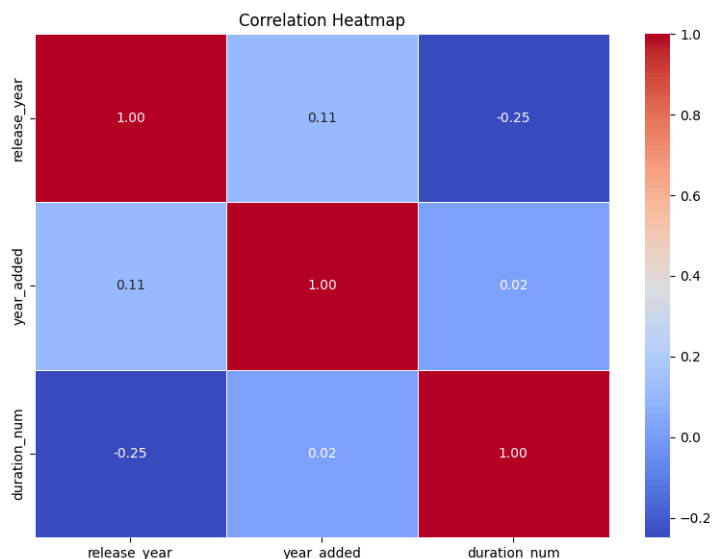
correlation_data = df[
    correlation_columns
].corr()

plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation_data,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    linewidths=0.5
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

```



STEP 12: TOP 10 COUNTRIES

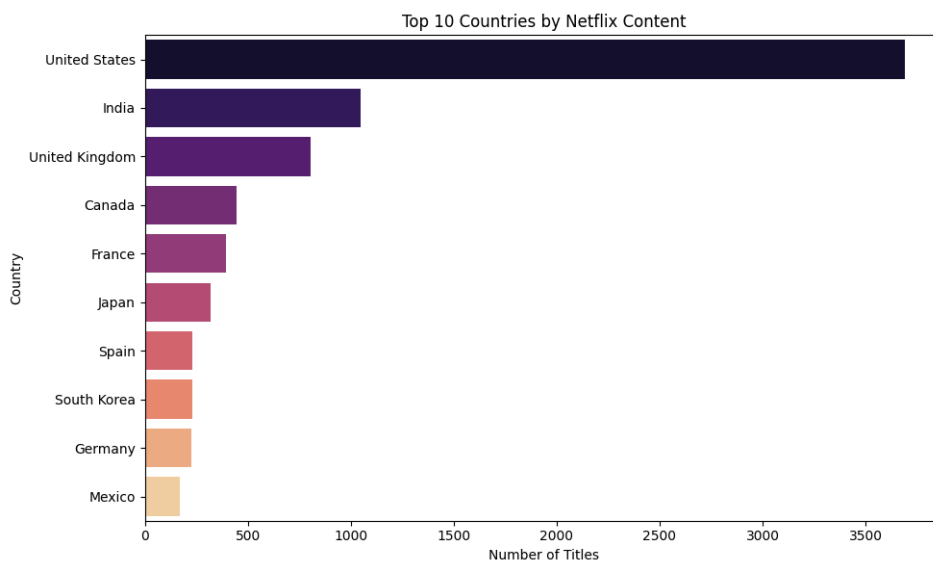
```
if "country" in df.columns:
```

```
    countries = (
        df["country"]
        .dropna()
        .str.split(", ")
        .explode()
        .value_counts()
        .head(10)
    )

    plt.figure(figsize=(10, 6))

    sns.barplot(
        x=countries.values,
        y=countries.index,
        hue=countries.index,
        palette="magma",
        legend=False
    )

    plt.title("Top 10 Countries by Netflix Content")
    plt.xlabel("Number of Titles")
    plt.ylabel("Country")
    plt.tight_layout()
    plt.show()
```



STEP 13: BUSINESS INSIGHTS

```
print("\n===== BUSINESS INSIGHTS =====")

if "type" in df.columns:

    most_common_type = df["type"].mode()[0]

    print("Most Common Content Type:", most_common_type)

if "rating" in df.columns:

    most_common_rating = df["rating"].mode()[0]

    print("Most Common Rating:", most_common_rating)

if "year_added" in df.columns:

    most_added_year = (
        df["year_added"]
        .dropna()
        .astype(int)
        .value_counts()
        .idxmax()
    )

    print("Year with Most Content Added:", most_added_year)

if "country" in df.columns:

    top_country = (
        df["country"]
        .dropna()
        .str.split(", ")
        .explode()
        .value_counts()
        .idxmax()
    )

    print("Country with Most Content:", top_country)
```

```
===== BUSINESS INSIGHTS =====
Most Common Content Type: Movie
```

QUESTIONS AND ANSWERS

Q1. Why is data visualization considered an essential component of data analytics?

Answer:

Data visualization converts raw numerical and categorical data into graphs and charts. It makes trends, patterns, relationships and outliers easier to identify and understand. It also helps organizations communicate insights and make data-driven decisions.

Q2. Differentiate between Matplotlib and Seaborn. Which library is more suitable for statistical visualizations and why?

Answer:

Matplotlib is a general-purpose visualization library that provides detailed control over graphs and charts.

Seaborn is built on Matplotlib and provides high-level functions for statistical visualizations.

Seaborn is generally more suitable for statistical visualizations because it provides simple functions for distributions, categorical analysis, relationships and correlation heatmaps.

Q3. Which type of chart would you choose to compare sales across different regions? Justify your answer.

Answer:

A bar chart is suitable for comparing values across different categories or regions.

Each category is represented by a separate bar, making it easy to identify the highest and lowest values.

In the Netflix dataset, a bar chart can similarly be used to compare the number of Movies and TV Shows or content by country.

Q4. What information can be obtained from a histogram and a box plot?

Answer:

A histogram displays the distribution and frequency of numerical values. It helps identify the shape and spread of data.

A box plot displays the median, quartiles, spread and possible outliers of numerical data.

In this experiment, the histogram shows release year distribution and the box plot shows content duration distribution.

Q5. Explain the purpose of a scatter plot. How does it help in identifying relationships between variables?

Answer:

A scatter plot displays individual observations using points based on two numerical variables.

It helps identify positive, negative or weak relationships between variables.

In this experiment, the scatter plot is used to analyze the relationship between release year and duration.

Q6. What is a correlation heatmap? How can it assist in feature selection for machine learning?

Answer:

A correlation heatmap is a graphical representation of correlations between numerical variables.

Values close to +1 indicate strong positive correlation, values close to -1 indicate strong negative correlation and values close to 0 indicate weak linear correlation.

It can help identify highly correlated or redundant features before applying machine learning algorithms.

Q7. Why is chart customization (titles, labels, legends, colors) important in data visualization?

Answer:

Chart customization improves readability and understanding.

Titles explain the purpose of the graph.

Axis labels identify the variables.

Legends explain different categories.

Colors help distinguish different groups.

Therefore, customization makes the visualization clearer and easier to interpret.

Q8. What factors should be considered while selecting an appropriate visualization for a dataset?

Answer:

The following factors should be considered:

1. Type of data
2. Number of variables
3. Analytical objective
4. Categorical or numerical data
5. Distribution of data
6. Relationship between variables
7. Time-based information
8. Audience and readability

The selected graph should clearly communicate the intended information.

Q9. How can misleading visualizations affect business decision-making? Give a real-world example.

Answer:

Misleading visualizations can make differences appear larger or smaller than they actually are and may result in incorrect business decisions.

For example, if a graph uses a truncated Y-axis, a small increase in Netflix content may appear to be a very large increase. This can cause management to make incorrect conclusions.

Q10. Explain how data visualization supports storytelling and business intelligence in organizations.

Answer:

Data visualization converts complex data into understandable visual stories.

Organizations can use charts and dashboards to identify trends, compare performance, discover problems and communicate insights.

For Netflix, visualizations can show content trends, popular ratings, content growth, country distribution and differences between Movies and TV Shows.

Therefore, data visualization supports business intelligence and data-driven decision-making.